CHEATSHEET · REFERENCIA RÁPIDA

Patrones de IA Agéntica

Qué es cada patrón, cuándo usarlo y cuándo no. Origen académico o industrial, y equivalencias de nomenclatura entre frameworks.

{dev/talles}

cursos.devtalles.com

01 **Agente único** 11 patrones

02 **Multi-agente** 9 patrones

03 **Control y seguridad** 6 patrones

04 **Contexto y memoria** 3 patrones

01 — PATRONES DE AGENTE ÚNICO

Tool Use

Toolformer 2023

El modelo invoca funciones externas para obtener datos o producir efectos que no puede generar solo. Patrón fundacional.

Úsalo: datos actuales, cálculo exacto, acciones sobre sistemas externos.

Planning

Andrew Ng

Descompone la tarea en pasos antes de ejecutar: un problema grande se vuelve una secuencia de problemas pequeños.

Úsalo: varios pasos con dependencias. **Evítalo:** tareas triviales — el overhead supera al beneficio.

Reflection

Self-Refine / Reflexion

El agente evalúa su propia salida y la refina iterativamente.

Límite: un modelo que se autoevalúa tiende a aprobarse. Exige rúbrica concreta o separar roles (Evaluator-Optimizer).

ReAct

Yao et al. 2022

Bucle que intercala razonamiento y acción: pensar → actuar → observar → pensar. Compone Planning + Tool Use.

Úsalo: el plan debe ajustarse sobre la marcha. **Riesgo:** bucle infinito sin condición de terminación.

Prompt Chaining

Anthropic

Cadena de pasos fijos con *gates* de validación opcionales entre ellos. El workflow más simple y, con frecuencia, el correcto.

Muchos sistemas montados como agentes funcionarían igual o mejor como cadena determinista.

Plan-and-Execute

Separa planificación y ejecución en dos fases, en vez del bucle intercalado de ReAct.

	ReAct	P&E
Llamadas	Muchas	Pocas
Coste	Alto	Bajo
Adaptabilidad	Alta	Baja

ReWOO

Reasoning WithOut Observation

Genera el plan completo **antes** de ejecutar herramientas, con variables placeholder para resultados futuros. Reduce mucho el consumo de tokens al no reinyectar el historial.

CodeAct

Wang et al. 2024

La acción es **código ejecutable** en lugar de llamadas JSON. Permite composición, bucles y condicionales: una acción hace lo que en JSON serían diez.

Requisito: sandboxing. No es opcional.

Self-Consistency

Generar N respuestas con temperatura alta y quedarse con la mayoritaria. Barato y efectivo en tareas con respuesta verificable o convergente.

Tree of Thoughts

Yao et al. 2023

Exploración ramificada con backtracking: cada nodo es un estado parcial que puede expandirse o abandonarse.

Potente pero muy caro; se usa poco fuera de benchmarks.

Agentic RAG

El agente decide **si** recuperar, **qué** consultar y **si** reformular — frente al pipeline fijo del RAG clásico.

02 — PATRONES MULTI-AGENTE

Router & Specialist

Routing · Anthropic

Un clasificador dirige cada consulta al especialista adecuado. Clasifica y despacha **una sola vez**; no retiene el control.

Fallo típico: fronteras mal definidas entre especialistas — el error está en las categorías, no en el modelo.

Handoff

OpenAI Agents SDK · MAF

Transferencia **dinámica** de control: el agente activo decide a quién pasar la tarea. Quien recibe el control lo retiene.

No es un workflow secuencial: allí el orden se fija de antemano (Prompt Chaining); aquí lo decide el agente en runtime.

Orchestrator-Workers

MAF: Magentic

Un agente central descompone la tarea dinámicamente, delega en workers y sintetiza. Mantiene el control todo el proceso.

vs. Router: el router se retira; el orquestador coordina de forma continua. **Coste:** el más alto de los multi-agente.

Parallelization

MAF: Concurrent

Sectioning: subtareas independientes en simultáneo. **Voting:** la misma tarea varias veces, agregando resultados.

Reduce latencia drásticamente. **Coste oculto:** las ramas no se conocen entre sí y pueden contradecirse — casi siempre exige una síntesis posterior o mejor aún, un agente coordinador que evite colisiones.

Group Chat / Debate

AutoGen

Varios agentes conversan en un hilo compartido; un orquestador elige quién habla (round-robin, por prompt o lógica custom).

Riesgo: bucles de cortesía infinita — se validan mutuamente sin converger.

Evaluator-Optimizer

Anthropic

Un agente genera, otro critica, se itera. Es Reflection con los roles en agentes distintos, lo que evita el autoengaño.

Requisito: rúbrica clara; sin ella el evaluador es tan complaciente como el generador.

LLM-as-a-Judge

Modelo que evalúa salidas contra una rúbrica explícita. Hace viable Evaluator-Optimizer, y también sirve solo para testing automatizado y CI.

Subagentes

Aislamiento de contexto

Delegar en instancias con ventana propia para que la exploración ruidosa no contamine el hilo principal. En la práctica, la razón principal para usar multi-agente.

Blackboard

IA simbólica, años 70-80

Agentes que colaboran escribiendo y leyendo un espacio compartido, sin hablarse entre ellos. Hoy el "blackboard" suele ser el sistema de archivos o una base de datos.

03 — CONTROL Y SEGURIDAD

Human-in-the-Loop

HITL

Intervención humana en puntos definidos: observador en group chat, revisor maker-checker, destino de escalado.

La decisión real no es cómo pausar, sino **dónde**. Un gate obligatorio vuelve el flujo síncrono: persiste estado en los checkpoints para reanudar sin repetir.

Approval Gates

HITL granular: aprobación explícita por acción irreversible, no supervisión continua. Autonomía en lo de bajo riesgo, parada en lo sensible.

Guardrails

Validación de entradas y salidas **fuera** del modelo: filtros de contenido, esquemas, dominio, detección de datos sensibles.

Lo primero que se exige en producción y a menudo lo último que se enseña.

Termination / Circuit Breaker

Límites duros de iteraciones, tiempo, coste o tokens. Nombre estándar en AutoGen y MAF de lo que algunos materiales llaman "Ejection".

Sin esto, cualquier bucle agéntico es un riesgo económico abierto.

Escalation / Fallback

Degradar a un modelo más simple, a lógica determinista o a un humano cuando la confianza cae bajo umbral o se agotan los reintentos.

Sandboxing / mínimo privilegio

Entorno aislado con permisos acotados a lo estrictamente necesario. Obligatorio con CodeAct; recomendable con cualquier Tool Use de escritura.

04 — CONTEXTO Y MEMORIA

Memory

Gestión de estado a través del tiempo. Uno de los factores más determinantes en producción y casi siempre ausente de los listados de patrones.

`Corto plazo`	Historial de la sesión actual
`Largo plazo`	Persistencia entre sesiones
`Episódica`	Interacciones concretas
`Semántica`	Hechos y conocimiento destilado

Compaction / Context Pruning

Resumir o podar el historial cuando la ventana se llena. Separa un agente de demo de uno que sobrevive sesiones largas.

Context Offloading

Sacar estado a archivos o almacenamiento externo y dejar solo referencias en el contexto: el "scratchpad en disco".

05 — NOMENCLATURA Y EQUIVALENCIAS

"Swarm" ≠ paralelización

Swarm es el framework experimental de OpenAI y su idea central son los **handoffs**. El patrón real de paralelización se llama **Parallelization** (Anthropic) o **Concurrent** (MAF).

"Ejection / Custom Logic"

El concepto es real —salir del bucle y devolver el control a código determinista— pero el nombre no es estándar. Busca: **Termination Conditions**, **Circuit Breaker**, **Escalation**, **Fallback**.

MICROSOFT AGENT FRAMEWORK → INDUSTRIA

MAF	Equivalente
`Sequential`	Prompt Chaining
`Concurrent`	Parallelization
`Handoff`	Handoff
`Group Chat`	Group Chat / Debate
`Magentic`	Orchestrator-Workers

MAF = convergencia de AutoGen y Semantic Kernel (Python y .NET). **Magentic** es el menos cableado a mano y el más caro: el manager LLM se invoca entre cada ejecución. **Aviso:** el paquete Python se reorganizó en 1.2.0 con cambios que rompen compatibilidad, ya después del 1.0.

06 — CRITERIO TRANSVERSAL Y RECURSOS

Empieza por la solución más simple y añade complejidad agéntica **solo cuando mejora resultados medibles**. Si la tarea se resuelve con una función, escribe la función. El error común es crear complejidad de coordinación cuando bastaba con sequential o concurrent.

Saber cuándo **no** aplicar un patrón vale tanto como saber implementarlo. — Anthropic, *Building Effective Agents* · Microsoft Azure Architecture Center

DOCUMENTACIÓN

Anthropic — Building Effective Agents	`anthropic.com/engineering/building-effective-agents`
Azure Architecture Center — Patrones	`learn.microsoft.com/…/ai-agent-design-patterns`
MAF — Overview	`learn.microsoft.com/en-us/agent-framework/overview/`
MAF — Orquestaciones	`…/agent-framework/workflows/orchestrations/`
MAF — Repositorio	`github.com/microsoft/agent-framework`
MAF — Blog del equipo	`devblogs.microsoft.com/agent-framework/`

PAPERS FUNDACIONALES

Tool Use	Toolformer — Schick et al., 2023
ReAct	Yao et al., 2022
Reflection	Self-Refine — Madaan et al., 2023
Reflection	Reflexion — Shinn et al., 2023
Tree of Thoughts	Yao et al., 2023
CodeAct	Wang et al., 2024
Orchestrator-Workers	Magentic-One — Microsoft Research, 2024

{dev/talles} cursos.devtalles.com →